



Jr. Full-Stack Developer — Technical Assessment

Overview

Octopi Digital is evaluating candidates for a Full-Stack Developer role through a hands-on technical assessment. You'll build a multi-tenant SaaS subscription platform from scratch — the kind of system our engineering team builds and maintains in production.

We care about your engineering judgment and ability to reason about real production concerns: data isolation, safe transactions, payment handling, and security. We're not testing typing speed or memorization.

Time: 2 days

Level: Jr. Full-Stack Developer

AI Usage

You're welcome to use any AI tool — Claude, ChatGPT, Copilot, Cursor, Gemini, agents, or anything else you normally work with. We don't care how much of the code you typed by hand. We care that you understand, can explain, and can modify every part of what you built. This will be tested directly in the review call.

What to Build

A web application where multiple organizations register, pay to activate, and each operates in a fully isolated space — its own users, subscription, and payment history. One organization must never be able to see or affect another organization's data.

- Organization
 - Users
 - Subscription
 - Payments
 - Transactions

1. Login & Access Control — Roles & Panels

Each role gets its own panel with clearly scoped permissions. Enforce all of this server-side — hiding buttons in the UI is not enough.

Platform Admin Panel

- Organizations list page: name, plan, status (active/trial/suspended/cancelled), member count, signup date — with search/filter
- Organization detail page: full org profile, its members, its subscription history, its payment/transaction history
- Plans management page: create/edit/disable plans — name, price, billing interval, feature list
- Transactions page (platform-wide): all transactions across all orgs, filterable by org/status/date
- Stats/overview page: total organizations, total users, active subscriptions, total revenue, failed payment count, recent signups
- Ability to suspend/reactivate any organization

Organization Admin Panel

- Org profile page: view/edit org name, contact info, billing email
- Members page: list of members with role and status, invite new member, remove member, change a member's role
- Subscription page: current plan, renewal date, usage/limits if applicable, upgrade/downgrade/cancel actions
- Billing & payments page: payment method management, payment history for their org only, download invoices
- Transactions page: their org's transactions only, with status filter

Organization Member Panel

- Own profile page: view/edit their own account details, change password
- Org info page: read-only view of the org they belong to (name, plan name — no billing/financial detail)
- No access to member management, billing, subscription changes, or transaction data

Also required

- Secure authentication (JWT or sessions), password hashing, forgot/reset password flow
- Protected routes on both frontend and backend
- Input validation and rate limiting on sensitive endpoints
- Session/token expiration handled correctly

2. Multi-Tenant Data Isolation

Data must be fully isolated between organizations, enforced at the API/database level. Document and justify your isolation approach in the README.

3. Registration & Payment

Organizations pay to complete registration — this is paid onboarding, not free signup.

Registration flow

- Signup page collects: organization name, admin's name, email, password, and plan selection
- After details are submitted, redirect to a checkout page (Stripe Checkout or Stripe Elements) showing: selected plan name, price, billing interval, and a "Pay & Activate" action
- On successful payment (confirmed via Stripe webhook — not just a frontend redirect), create the organization and activate the subscription as ACTIVE
- On failed/abandoned payment, the organization should not be created as active — leave it in a PENDING or not-created state, with an option to retry checkout

Required tool: Stripe (test mode) for checkout and webhook-based payment confirmation. Always verify payment success server-side via webhook — never trust a frontend redirect alone.

Ongoing subscription payments

- Plan selection and checkout
- Success and failure states handled correctly
- Payment history per organization
- Subscription status tracking: ACTIVE, PENDING, FAILED, CANCELLED, EXPIRED
- No duplicate processing of the same payment

4. Safe Database Transactions

Payment confirmation must update multiple related records (payment, subscription, organization) as a single unit — if any step fails, nothing should be left partially updated. The same webhook arriving more than once must not cause duplicate effects.

5. Payment Security

- Never store card numbers, CVV, or full payment credentials
- Verify webhook authenticity
- Verify every payment server-side — never trust the frontend alone
- Secrets stored in environment variables, not in code

6. Email Notifications

Each organization should receive notification emails for key events. Use any provider/service you're comfortable with (e.g. Nodemailer with a test SMTP service, SendGrid, Resend, etc.) — a test/sandbox mode is fine, no need for a real production email service.

Trigger notifications for

- New member invited to an organization
- Payment succeeded / payment failed
- Subscription upgraded, downgraded, or cancelled
- Subscription expiring soon (reminder)

7. Transaction History Page

A table of transactions with status: PENDING, SUCCESS, FAILED, REFUNDED, ROLLED_BACK.

8. Tech Stack

- Frontend: React or Next.js, with Redux Toolkit and/or TanStack Query (React Query) for state and data fetching, responsive UI, proper loading/error/empty states
- Backend: Node.js with Express or NestJS, REST API or GraphQL
- Database: MongoDB or PostgreSQL — properly designed schema, relationships, and indexes
- Payments: Stripe (test mode)
- Git with a real commit history

9. Error Handling

Handle gracefully: invalid login, expired session, unauthorized access, wrong-tenant access, payment failure, duplicate payment, webhook failure, failed transaction, network errors, bad form input. Show useful messages without leaking backend details.

10. Tests

Cover at minimum: authentication, role authorization, tenant isolation, the payment flow, duplicate webhook handling, and transaction rollback. Full coverage isn't the goal — proving you know what matters to test is.

Bonus (Optional — Not Required to Pass)

Extra credit only. Skip if you're tight on time; the core task above is evaluated first.

- CI/CD: a working GitHub Actions pipeline that automatically runs linting/tests on every push, and builds the project successfully. No live deployment or server required — a green pipeline that proves the app builds and tests pass cleanly earns this bonus.
- UI polish: thoughtful visual design — good spacing, hierarchy, consistent components, a design system feel rather than default styling
- Billing invoice generation: generate a PDF invoice per payment (using Puppeteer or similar), including organization name, plan name, billing period/duration, amount charged, payment date, and a unique invoice number — downloadable from the org's billing page

- Custom email configuration per organization: let each Organization Admin configure their own outgoing email (SMTP host/port, email address, and password or app password — e.g. for Gmail) in a settings page, so notification emails for their org are sent from their own email address instead of a shared platform email. Store credentials securely (encrypted at rest, never returned in plaintext via the API).

Important Note — Submit Even If Incomplete

Don't worry if you're not able to finish every part of the task in the given time. We evaluate your effort, thought process, and engineering decisions — not just whether the checklist is 100% complete. A partially finished project with clear thinking and solid fundamentals is far more valuable to us than rushing to tick every box. Submit what you have by the deadline, and be honest in your README about what's done, what's partial, and what you'd do next with more time.

What to Submit

1. Source Code — a Git repo with frontend/ and backend/ folders and a README.md. Structure can vary.
2. README — cover: architecture, tech choices, database design, multi-tenant approach, auth strategy, payment flow, transaction/rollback approach, security notes, how you used AI tools, how to run it locally, required env variables, known limitations.
3. Test Credentials — provide working login credentials for all three roles (Platform Admin, Organization Admin, Organization Member) so we can log in and test each role directly.
4. Two Short Videos — Strict 5-Minute Limit Each

Video 1 — Product walkthrough (max 5 minutes, no code): Show the working app: register an org, log in with different roles, subscribe, pay, view dashboards and transaction history. This is a demo, not a code review.

Video 2 — Code walkthrough (max 5 minutes, code only): Walk through the code structure and the major functionality — how auth works, how tenant isolation is enforced, how the transaction/rollback logic works, how webhooks are handled.

Both videos are hard-capped at 5 minutes. Going over is a mark against you — practice getting to the point.

How It's Evaluated

Area	Weight
Architecture (clean, scalable, well-separated)	20%
Backend & database (APIs, schema, transactions, rollback)	20%
Multi-tenancy (real isolation, no data leaks)	15%
Payments (workflow, webhooks, idempotency, failure handling)	15%
Security (auth, validation, secrets, payment handling)	10%
Frontend (UI, state management, responsiveness)	10%
Code quality & docs (readability, Git history, tests)	10%

One Rule

Build exactly what's described above — nothing more. Don't add extra features, screens, or "nice to have" ideas beyond the core task and the listed bonuses. Extra unrequested work doesn't earn credit and isn't what's being evaluated.
